

2.2. Writing CUDA SIMT Kernels

CUDA C++ kernels can largely be written in the same way that traditional CPU code would be written for a given problem. However, there are some **unique features of the GPU that can be used to improve performance**. Additionally, some understanding of how threads on the GPU are scheduled, how they access memory, and how their execution proceeds can help developers write kernels that maximize utilization of the available computing resources.

2.2.1. Basics of SIMT

From the developer’s perspective, the CUDA thread is the fundamental unit of parallelism. Warps and SIMT describes the basic SIMT model of GPU execution and SIMT Execution Model provides additional details of the SIMT model. The **SIMT model allows each thread to maintain its own state and control flow**. From a functional perspective, each thread can execute a separate code path. However, substantial performance improvements can be realized by taking care that kernel code minimizes the situations where threads in the same warp take divergent code paths.

2.2.2. Thread Hierarchy

Threads are organized into thread blocks, which are then organized into a grid. Grids may be 1, 2, or 3 dimensional and the size of the grid can be queried inside a kernel with the `gridDim` built-in variable. Thread blocks may also be 1, 2, or 3 dimensional. The size of the thread block can be queried inside a kernel with the `blockDim` built-in variable. The index of the thread block can be queried with the `blockIdx` built-in variable. Within a thread block, the index of the thread is obtained using the `threadIdx` built-in variable. These built-in variables are used to compute a unique global thread index for each thread, thereby enabling each thread to load/store specific data from global memory and execute a unique code path as needed.

- `gridDim.[x|y|z]`: Size of the grid in the `x`, `y` and `z` dimension respectively. These values are set at kernel launch.
- `blockDim.[x|y|z]`: Size of the block in the `x`, `y` and `z` dimension respectively. These values are set at kernel launch.
- `blockIdx.[x|y|z]`: Index of the block in the `x`, `y` and `z` dimension respectively. These values change depending on which block is executing.
- `threadIdx.[x|y|z]`: Index of the thread in the `x`, `y` and `z` dimension respectively. These values change depending on which thread is executing.

The use of **multi-dimensional thread blocks and grids** is **for convenience only** and **does not affect performance**. The threads of a block are linearized predictably: the first index `x` moves the fastest, followed by `y` and then `z`. This means that in the linearization of a thread indices, consecutive **values of `threadIdx.x` indicate consecutive threads**, **`threadIdx.y` has a stride of `blockDim.x`**, and **`threadIdx.z` has a stride of `blockDim.x * blockDim.y`**. This affects how threads are assigned to warps, as detailed in Hardware Multithreading.

Figure 9 shows a simple example of a 2D grid, with 1D thread blocks.

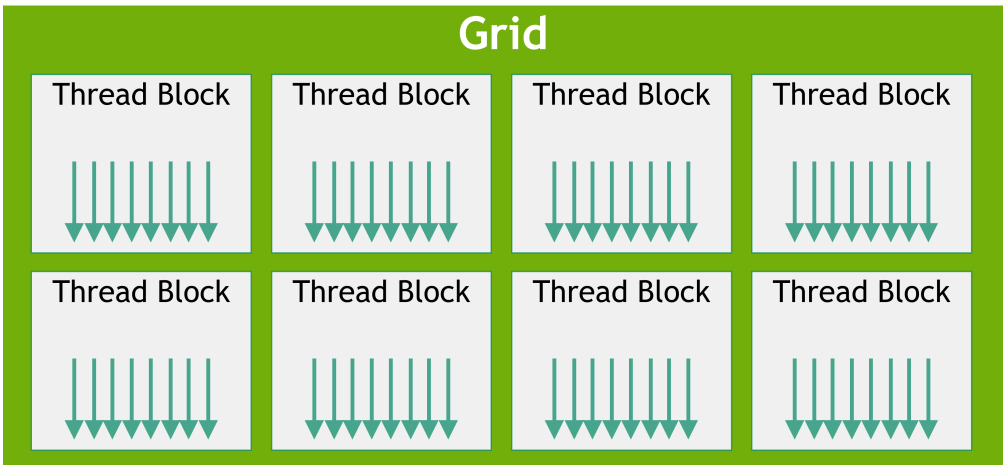


Figure 9: Grid of Thread Blocks

2.2.3. GPU Device Memory Spaces

CUDA devices have **several memory spaces** that can be accessed by CUDA threads within kernels. [Table 1](#) shows a summary of the common memory types, their thread scopes, and their lifetimes. The following sections explain each of these memory types in more detail.

Table 1: Memory Types, Scopes and Lifetimes

Memory Type	Scope	Lifetime	Location
Global	Grid	Application	Device
Constant	Grid	Application	Device
Shared	Block	Kernel	SM
Local	Thread	Kernel	Device
Register	Thread	Kernel	SM

2.2.3.1. Global Memory

Global memory (also called device memory) is the **primary memory space for storing data that is accessible by all threads in a kernel**. It is similar to RAM in a CPU system. Kernels running on the GPU have direct access to global memory in the same way code running on the CPU has access to system memory.

Global memory is persistent. That is, an allocation made in global memory and the data stored in it **persist until the allocation is freed or until the application is terminated**. `cudaDeviceReset` also frees all allocations.

Global memory is allocated with CUDA API calls such as `cudaMalloc` and `cudaMallocManaged`. **Data can be copied into global memory from CPU memory** using CUDA runtime API calls such as `cudaMemcpy`. Global memory allocations made with CUDA APIs are freed using `cudaFree`.

Prior to a kernel launch, global memory is allocated and initialized by CUDA API calls. During kernel execution, data from global memory can be read by the CUDA threads, and the result from operations carried out by CUDA threads can be written back to global memory. Once a kernel has completed execution, the results it wrote to global memory can be copied back to the host or used by other kernels on the GPU.

Because global memory is accessible by all threads in a grid, care must be taken to **avoid data races between threads**. Since **CUDA kernels** launched from the host **have the return type `void`**, **the only way for numerical results computed by a kernel to be returned to the host is by writing those results to global memory**.

A simple example illustrating the use of global memory is the `vecAdd` kernel below, where the three arrays `A`, `B`, and `C` are in global memory and are being accessed by this vector add kernel.

```
__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}
```

2.2.3.2. Shared Memory

Shared memory is a memory space that is **accessible by all threads in a thread block**. It is **physically located on each SM** and uses the same physical resource as the L1 cache, the unified data cache. The data in shared memory persists throughout the kernel execution. Shared memory can be considered a user-managed scratchpad for use during kernel execution. While small in size compared to global memory, because shared memory is located on each SM, **the bandwidth is higher and the latency is lower than accessing global memory**.

Since shared memory is accessible by all threads in a thread block, care must be taken to **avoid data races between threads in the same thread block**. **Synchronization** between threads in the same thread block can be achieved **using the**

`__syncthreads()` function. This function **blocks all threads in the thread block until all threads have reached the call to** `__syncthreads()`.

```
// assuming blockDim.x is 128
__global__ void example_syncthreads(int* input_data, int* output_data) {
    __shared__ int shared_data[128];
    // Every thread writes to a distinct element of 'shared_data':
    shared_data[threadIdx.x] = input_data[threadIdx.x];

    // All threads synchronize, guaranteeing all writes to 'shared_data' are ordered
    // before any thread is unblocked from '__syncthreads()':
    __syncthreads();

    // A single thread safely reads 'shared_data':
    if (threadIdx.x == 0) {
        int sum = 0;
        for (int i = 0; i < blockDim.x; ++i) {
            sum += shared_data[i];
        }
        output_data[blockIdx.x] = sum;
    }
}
```

The size of shared memory varies depending on the GPU architecture being used. Because **shared memory and L1 cache share the same physical space**, **using shared memory reduces the size of the usable L1 cache** for a kernel. Additionally, if no shared memory is used by the kernel, the entire physical space will be utilized by L1 cache. The CUDA runtime API provides functions to query the shared memory size on a per SM basis and a per thread block basis, using the `cudaGetDeviceProperties` function and investigating the `cudaDeviceProp.sharedMemPerMultiprocessor` and `cudaDeviceProp.sharedMemPerBlock` device properties.

The CUDA runtime API provides a function `cudaFuncSetCacheConfig` to tell the runtime whether to allocate more space to shared memory, or more space to L1 cache. This function specifies a preference to the runtime, but is not guaranteed to be honored. The runtime is free to make decisions based on the available resources and the needs of the kernel.

Shared memory can be allocated both statically and dynamically.

2.2.3.2.1. Static Allocation of Shared Memory

To **allocate shared memory statically**, the programmer must **declare a variable inside the kernel using the** `__shared__` specifier. The variable will be allocated in shared memory and will persist for the duration of the kernel execution. **The size of the shared memory declared in this way must be specified at compile time**. For example, the following code snippet, located in the body of the kernel, declares a shared memory array of type `float` with 1024 elements.

```
__shared__ float sharedArray[1024];
```

After this declaration, all the threads in the thread block will have access to this shared memory array. Care must be taken to avoid data races between threads in the same thread block, typically with the use of `__syncthreads()`.

2.2.3.2.2. Dynamic Allocation of Shared Memory

To **allocate shared memory dynamically**, the programmer can specify the desired amount of shared memory per thread block in bytes as the third (and optional) argument to the kernel launch in the triple chevron notation like this

```
functionName<<<grid, block, sharedMemoryBytes>>>().
```

Then, inside the kernel, the programmer can **use the** `extern __shared__` **specifier to declare a variable** that will be allocated dynamically at kernel launch.

```
extern __shared__ float sharedArray[];
```

One caveat is that if one wants multiple dynamically allocated shared memory arrays, the single `extern __shared__` must be partitioned manually using pointer arithmetic. For example, if one wants the equivalent of the following,

```
short array0[128];
float array1[64];
int   array2[256];
```

in dynamically allocated shared memory, one could declare and initialize the arrays in the following way:

```
extern __shared__ float array[];

short* array0 = (short*)array;
float* array1 = (float*)&array0[128];
int*   array2 = (int*)&array1[64];
```

Note that pointers need to be aligned to the type they point to, so the following code, for example, does not work since `array1` is not aligned to 4 bytes.

```
extern __shared__ float array[];
short* array0 = (short*)array;
float* array1 = (float*)&array0[127];
```

2.2.3.3. Registers

Registers are located on the SM and have thread local scope. Register usage is managed by the compiler and registers are used for thread local storage during the execution of a kernel. The number of registers per SM and the number of registers per thread block can be queried using the `regsPerMultiprocessor` and `regsPerBlock` device properties of the GPU.

NVCC allows the developer to specify a maximum number of registers to be used by a kernel via the `-maxrregcount` option. Using this option to reduce the number of registers a kernel can use may result in more thread blocks being scheduled on the SM concurrently, but may also result in more register spilling.

2.2.3.4. Local Memory

Local memory is thread local storage **similar to registers** and managed by NVCC, but the **physical location of local memory is in the global memory space**. The ‘local’ label refers to its logical scope, not its physical location. Local memory is used for thread local storage during the execution of a kernel. **Automatic variables that the compiler is likely to place in local memory are:**

- Arrays for which it cannot determine that they are indexed with constant quantities,
- Large structures or arrays that would consume too much register space,
- Any variable if the kernel **uses more registers than available, that is register spilling**.

Because the local memory space resides in device memory, local memory accesses have the same latency and bandwidth as global memory accesses and are subject to the same requirements for memory coalescing as described in [Coalesced Global Memory Access](#). Local memory is however organized such that consecutive 32-bit words are accessed by consecutive thread IDs. Accesses are therefore fully coalesced as long as all threads in a warp access the same relative address, such as the same index in an array variable or the same member in a structure variable.

2.2.3.5. Constant Memory

Constant memory has a `grid scope` and is accessible for the lifetime of the application. The constant memory resides on the device and is `read-only to the kernel`. As such, it `must be declared and initialized on the host with the __constant__ specifier`, outside any function.

The `__constant__` memory space specifier declares a variable that:

- Resides in constant memory space,
- Has the lifetime of the CUDA context in which it is created,
- Has a distinct object per device,
- Is accessible from all the threads within the grid and from the host through the runtime library (`cudaGetSymbolAddress()` / `cudaGetSymbolSize()` / `cudaMemcpyToSymbol()` / `cudaMemcpyFromSymbol()`).

The total amount of constant memory can be queried with the `totalConstMem` device property element.

Constant memory is `useful for small amounts of data` that each thread will `use in a read-only fashion`. Constant memory is small relative to other memories, typically 64KB per device.

An example snippet of declaring and using constant memory follows.

```
// In your .cu file
__constant__ float coeffs[4];

__global__ void compute(float *out) {
    int idx = threadIdx.x;
    out[idx] = coeffs[0] * idx + coeffs[1];
}

// In your host code
float h_coeffs[4] = {1.0f, 2.0f, 3.0f, 4.0f};
cudaMemcpyToSymbol(coeffs, h_coeffs, sizeof(h_coeffs));
compute<<<1, 10>>>(device_out);
```

2.2.3.6. Caches

GPU devices have a `multi-level cache structure which includes L2 and L1 caches`.

The L2 cache is located on the device and is shared among all the SMs. The size of the L2 cache can be queried with the `L2CacheSize` device property element from the function `cudaGetDeviceProperties`.

As described above in [Shared Memory](#), `L1 cache is physically located on each SM` and is the same physical space used by shared memory. If no shared memory is utilized by a kernel, the entire physical space will be utilized by the L1 cache.

The L2 and L1 caches can be controlled via functions that allow the developer to specify various caching behaviors. The details of these functions are found in [Configuring L1/Shared Memory Balance](#), [L2 Cache Control](#), and [Low-Level Load and Store Functions](#).

If these hints are not used, the compiler and runtime will do their best to utilize the caches efficiently.

2.2.3.7. Texture and Surface Memory

Note

Some older CUDA code may use texture memory because, in older NVIDIA GPUs, doing so provided performance benefits in some scenarios. On all currently supported GPUs, these scenarios may be handled using direct load and store instructions, and use of texture and surface memory instructions no longer provides any performance benefit.

A GPU may have specialized instructions for loading data from an image to be used as textures in 3D rendering. CUDA exposes these instructions and the machinery to use them in the [texture object API](#) and the [surface object API](#).

Texture and Surface memory are not discussed further in this guide as there is no advantage to using them in CUDA on any currently supported NVIDIA GPU. CUDA developers should feel free to ignore these APIs. For developers working on existing code bases which still use them, explanations of these APIs can still be found in the legacy [CUDA C++ Programming Guide](#).

2.2.3.8. Distributed Shared Memory

[Thread Block Clusters](#) introduced in compute capability 9.0 and facilitated by [Cooperative Groups](#), provide the ability for threads in a thread block cluster to [access shared memory of all the participating thread blocks in that cluster](#). This [partitioned shared memory is called *Distributed Shared Memory*](#), and the corresponding [address space is called Distributed Shared Memory address space](#). Threads that belong to a thread block cluster [can read, write or perform atomics](#) in the distributed address space, regardless whether the address belongs to the local thread block or a remote thread block. Whether a kernel uses distributed shared memory or not, the shared memory size specifications, static or dynamic is still per thread block. The size of distributed shared memory is just the number of thread blocks per cluster multiplied by the size of shared memory per thread block.

[Accessing data in distributed shared memory requires all the thread blocks to exist](#). A user can guarantee that all thread blocks have started executing using `cluster.sync()` from [class cluster_group](#). The user [also needs to ensure that all distributed shared memory operations happen before the exit of a thread block](#), e.g., if a remote thread block is trying to read a given thread block's shared memory, the program needs to ensure that the shared memory read by the remote thread block is completed before it can exit.

Let's look at a [simple histogram computation](#) and how to optimize it on the GPU using thread block cluster. A standard way of computing histograms is to perform the computation in the shared memory of each thread block and then perform global memory atomics. A limitation of this approach is the shared memory capacity. Once the histogram bins no longer fit in the shared memory, a user needs to directly compute histograms and hence the atomics in the global memory. With distributed shared memory, CUDA provides an intermediate step, where depending on the histogram bins size, the histogram can be computed in shared memory, distributed shared memory or global memory directly.

[The CUDA kernel example below shows how to compute histograms](#) in shared memory or distributed shared memory, depending on the number of histogram bins.

```

#include <cooperative_groups.h>

// Distributed Shared memory histogram kernel
__global__ void clusterHist_kernel(int *bins, const int nbins, const int bins_per_block, const int *__restrict,
                                   size_t array_size)
{
    extern __shared__ int smem[];
    namespace cg = cooperative_groups;
    int tid = cg::this_grid().thread_rank();

    // Cluster initialization, size and calculating local bin offsets.
    cg::cluster_group cluster = cg::this_cluster();
    unsigned int clusterBlockRank = cluster.block_rank();
    int cluster_size = cluster.dim_blocks().x;

    for (int i = threadIdx.x; i < bins_per_block; i += blockDim.x)
    {
        smem[i] = 0; //Initialize shared memory histogram to zeros
    }

    // cluster synchronization ensures that shared memory is initialized to zero in
    // all thread blocks in the cluster. It also ensures that all thread blocks
    // have started executing and they exist concurrently.
    cluster.sync();

    for (int i = tid; i < array_size; i += blockDim.x * gridDim.x)
    {
        int ldata = input[i];

        //Find the right histogram bin.
        int binid = ldata;
        if (ldata < 0)
            binid = 0;
        else if (ldata >= nbins)
            binid = nbins - 1;

        //Find destination block rank and offset for computing
        //distributed shared memory histogram
        int dst_block_rank = (int)(binid / bins_per_block);
        int dst_offset = binid % bins_per_block;

        //Pointer to target block shared memory
        int *dst_smem = cluster.map_shared_rank(smem, dst_block_rank);

        //Perform atomic update of the histogram bin
        atomicAdd(dst_smem + dst_offset, 1);
    }

    // cluster synchronization is required to ensure all distributed shared
    // memory operations are completed and no thread block exits while
    // other thread blocks are still accessing distributed shared memory
    cluster.sync();

    // Perform global memory histogram, using the local distributed memory histogram
    int *lbins = bins + cluster.block_rank() * bins_per_block;
    for (int i = threadIdx.x; i < bins_per_block; i += blockDim.x)
    {
        atomicAdd(&lbins[i], smem[i]);
    }
}

```

The above kernel can be launched at runtime with a cluster size depending on the amount of distributed shared memory required. If the histogram is small enough to fit in shared memory of just one block, the user can launch the kernel with cluster size 1. The code snippet below shows how to launch a cluster kernel dynamically based on shared memory requirements.

```
// Launch via extensible launch
{
    cudaLaunchConfig_t config = {0};
    config.gridDim = array_size / threads_per_block;
    config.blockDim = threads_per_block;

    // cluster_size depends on the histogram size.
    // ( cluster_size == 1 ) implies no distributed shared memory, just thread block local shared memory
    int cluster_size = 2; // size 2 is an example here
    int nbins_per_block = nbins / cluster_size;

    //dynamic shared memory size is per block.
    //Distributed shared memory size = cluster_size * nbins_per_block * sizeof(int)
    config.dynamicSmemBytes = nbins_per_block * sizeof(int);

    CUDA_CHECK(cudaFuncSetAttribute((void *)clusterHist_kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
    config.dynamicSmemBytes));

    cudaLaunchAttribute attribute[1];
    attribute[0].id = cudaLaunchAttributeClusterDimension;
    attribute[0].val.clusterDim.x = cluster_size;
    attribute[0].val.clusterDim.y = 1;
    attribute[0].val.clusterDim.z = 1;

    config.numAttrs = 1;
    config.attrs = attribute;

    cudaLaunchKernelEx(&config, clusterHist_kernel, bins, nbins, nbins_per_block, input, array_size);
}
```

2.2.4. Memory Performance

Ensuring proper memory usage is key to achieving high performance in CUDA kernels. This section discusses some general principles and examples for achieving high memory throughput in CUDA kernels.

2.2.4.1. Coalesced Global Memory Access

Global memory is accessed via 32-byte memory transactions. When a CUDA thread requests a word of data from global memory, the relevant warp coalesces the memory requests from all the threads in that warp into the number of memory transactions necessary to satisfy the request, depending on the size of the word accessed by each thread and the distribution of the memory addresses across the threads. For example, if a thread requests a 4-byte word, the actual memory transaction the warp will generate to global memory will be 32 bytes in total. To use the memory system most efficiently, the warp should use all the memory that is fetched in a single memory transaction. That is, if a thread is requesting a 4-byte word from global memory, and the transaction size is 32 bytes, if other threads in that warp can use other 4-byte words of data from that 32-byte request, this will result in the most efficient use of the memory system.

As a simple example, if consecutive threads in the warp request consecutive 4-byte words in memory, then the warp will request 128 bytes of memory total, and this 128 bytes required will be fetched in four 32-byte memory transactions. This results in 100% utilization of the memory system. That is, 100% of the memory traffic is utilized by the warp. Figure 10 illustrates this example of perfectly coalesced memory access.

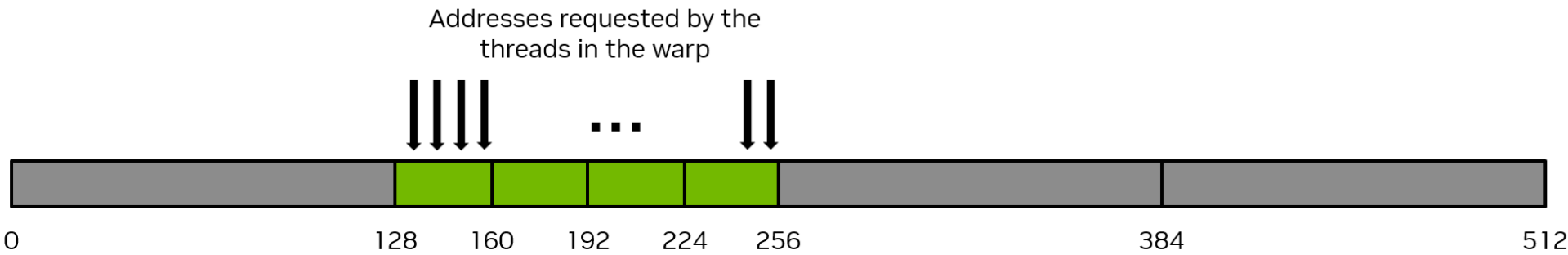


Figure 10: Coalesced memory access

Conversely, the pathologically worst case scenario is when consecutive threads access data elements that are 32 bytes or more apart from each other in memory. In this case, the warp will be forced to issue a 32-byte memory transaction for each thread, and the total number of bytes of memory traffic will be 32 bytes times 32 threads/warp = 1024 bytes. However, the amount of memory used will be 128 bytes only (4 bytes for each thread in the warp), so the memory utilization will only be $128 / 1024 = 12.5\%$. This is a very inefficient use of the memory system. Figure 11 illustrates this example of uncoalesced memory access.

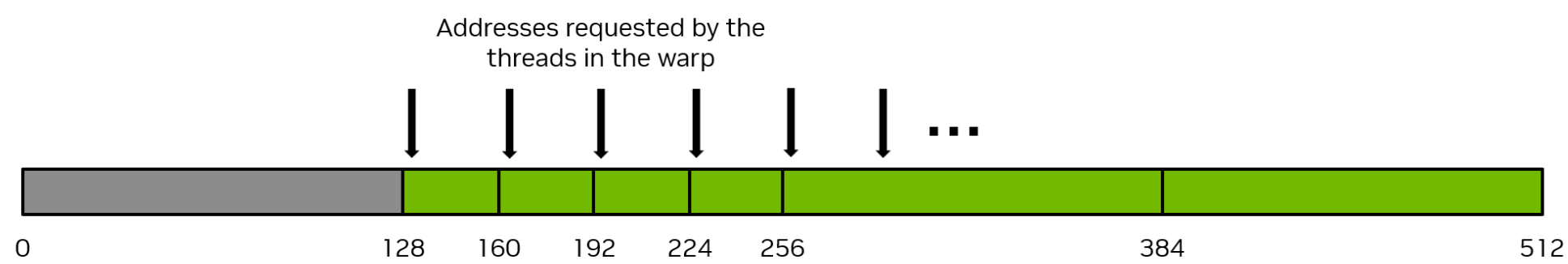


Figure 11: Uncoalesced memory access

The most straightforward way to achieve coalesced memory access is for consecutive threads to access consecutive elements in memory. For example, for a kernel launched with 1d thread blocks, the following VecAdd kernel will achieve coalesced memory access. Notice how thread workIndex accesses the three arrays, and consecutive threads (indicated by consecutive values of workIndex) access consecutive elements in the arrays.

```
__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    int workIndex = threadIdx.x + blockIdx.x*blockDim.x;
    if(workIndex < vectorLength)
    {
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}
```

There is no requirement that consecutive threads access consecutive elements of memory to achieve coalesced memory access, it is merely the typical way coalescing is achieved. Coalesced memory access occurs provided all the threads in the warp access elements from the same 32-byte segments of memory in some linear or permuted way. Stated another way, the best way to achieve coalesced memory access is to maximize the ratio of bytes used to bytes transferred.

i **Note**

Ensuring proper coalescing of global memory accesses is one of the most important performance considerations for writing performant CUDA kernels. It is imperative that applications use the memory system as efficiently as possible.

2.2.4.1.1. Matrix Transpose Example Using Global Memory

As a simple example, consider an out-of-place matrix transpose kernel that transposes a 32 bit float square matrix of size N x N, from matrix a to matrix c. This example uses a 2d grid, and assumes a launch of 2d thread blocks of size 32 x 32 threads, that is, blockDim.x = 32 and blockDim.y = 32, so each 2d thread block will operate on a 32 x 32 tile of the matrix. Each thread operates on a unique element of the matrix, so no explicit synchronization of threads is necessary. Figure 12 illustrates this matrix transpose operation. The kernel source code follows the figure.

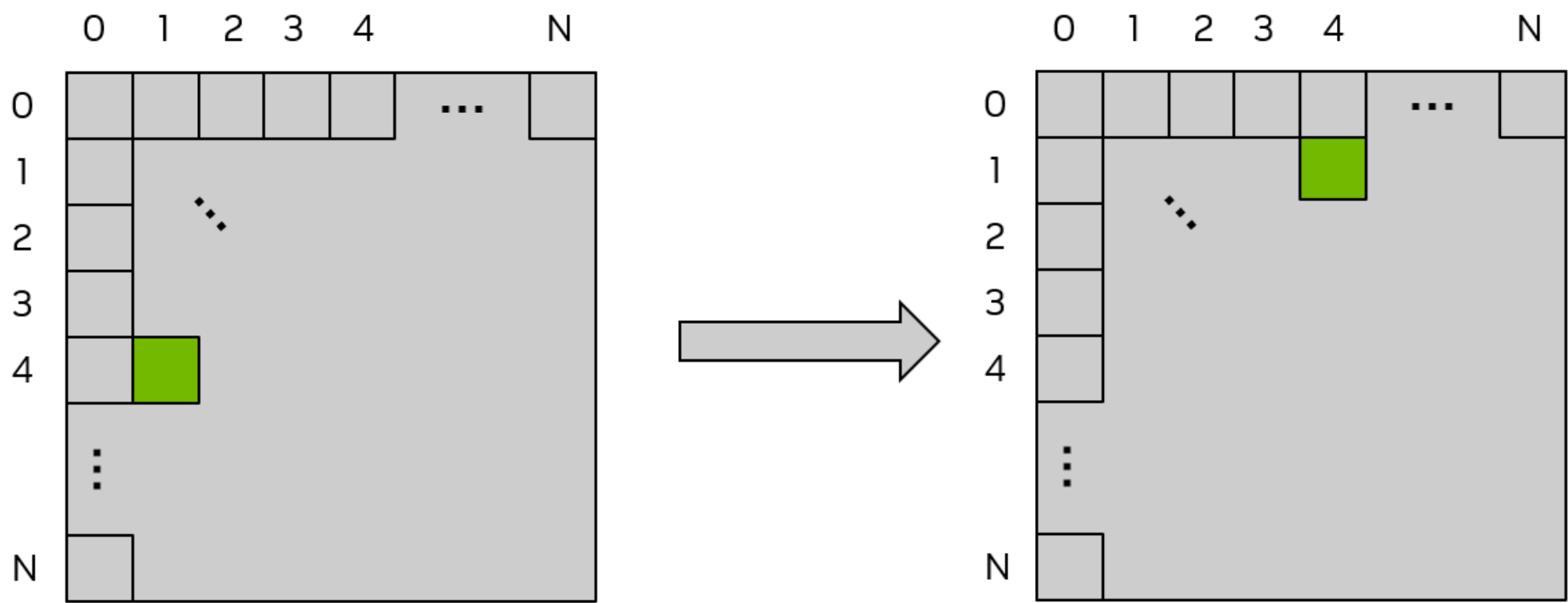


Figure 12: Matrix Transpose using Global memory

The labels on the top and left of each matrix are the 2d thread block indices and also can be considered the tile indices, where each small square indicates a tile of the matrix that will be operated on by a 2d thread block. In this example, the tile size is 32 x 32 elements, so each of the small squares represents a 32 x 32 tile of the matrix. The green shaded square shows the location of an example tile before and after the transpose operation.

```
/* macro to index a 1D memory array with 2D indices in row-major order */
/* ld is the leading dimension, i.e. the number of columns in the matrix */
#define INDX( row, col, ld ) ( ( row ) * (ld) ) + (col) )

/* CUDA kernel for naive matrix transpose */
__global__ void naive_cuda_transpose(int m, float *a, float *c )
{
    int myCol = blockDim.x * blockIdx.x + threadIdx.x;
    int myRow = blockDim.y * blockIdx.y + threadIdx.y;

    if( myRow < m && myCol < m )
    {
        c[INDX( myCol, myRow, m )] = a[INDX( myRow, myCol, m )];
    } /* end if */
    return;
} /* end naive_cuda_transpose */
```

To determine whether this kernel is achieving coalesced memory access one needs to determine whether consecutive threads are accessing consecutive elements of memory. In a 2d thread block, the `x` index moves the fastest, so consecutive values of `threadIdx.x` should be accessing consecutive elements of memory. `threadIdx.x` appears in `myCol`, and one can observe that when `myCol` is the second argument to the `INDX` macro, consecutive threads are reading consecutive values of `a`, so the read of `a` is perfectly coalesced.

However, the writing of `c` is not coalesced, because consecutive values of `threadIdx.x` (again examine `myCol`) are writing elements to `c` that are `ld` (leading dimension) elements apart from each other. This is observed because now `myCol` is the first argument to the `INDX` macro, and as the first argument to `INDX` increments by 1, the memory location changes by `ld`. When `ld` is larger than 32 (which occurs whenever the matrix sizes are larger than 32), this is equivalent to the pathological case shown in Figure 11.

To alleviate these uncoalesced writes, the use of shared memory can be employed, which will be described in the next section.

2.2.4.2. Shared Memory Access Patterns

Shared memory has 32 banks that are organized such that successive 32-bit words map to successive banks. Each bank has a bandwidth of 32 bits per clock cycle.

When multiple threads in the same warp attempt to access different elements in the same bank, a bank conflict occurs. In this case, the access to the data in that bank will be serialized until the data in that bank has been obtained by all the threads that have requested it. This serialization of access results in a performance penalty.

The two exceptions to this scenario happen when multiple threads in the same warp are accessing (either reading or writing) the same shared memory location. For read accesses, the word is broadcast to the requesting threads. For write accesses, each shared memory address is written by only one of the threads (which thread performs the write is undefined).

Figure 13 shows some examples of strided access. The red box inside the bank indicates a unique location in shared memory.

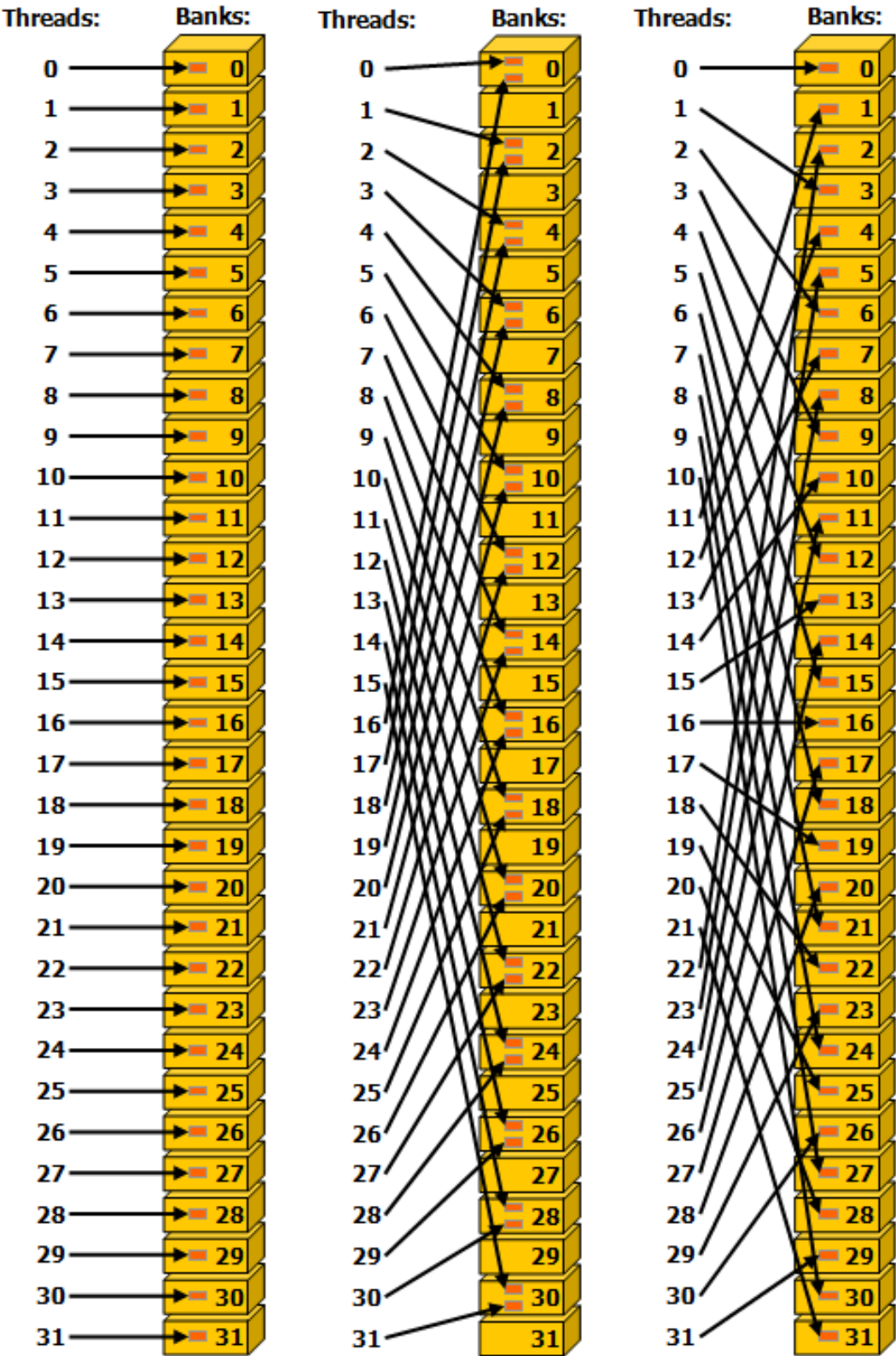


Figure 13: Strided Shared Memory Accesses in 32 bit bank size mode.

Left

Linear addressing with a stride of one 32-bit word (no bank conflict).

Middle

Linear addressing with a stride of two 32-bit words (two-way bank conflict).

Right

Linear addressing with a stride of three 32-bit words (no bank conflict).

Figure 14 shows some examples of memory read accesses that involve the broadcast mechanism. The red box inside the bank indicates a unique location in shared memory. If multiple arrows point to the same location, the data is broadcast to all threads that requested it.

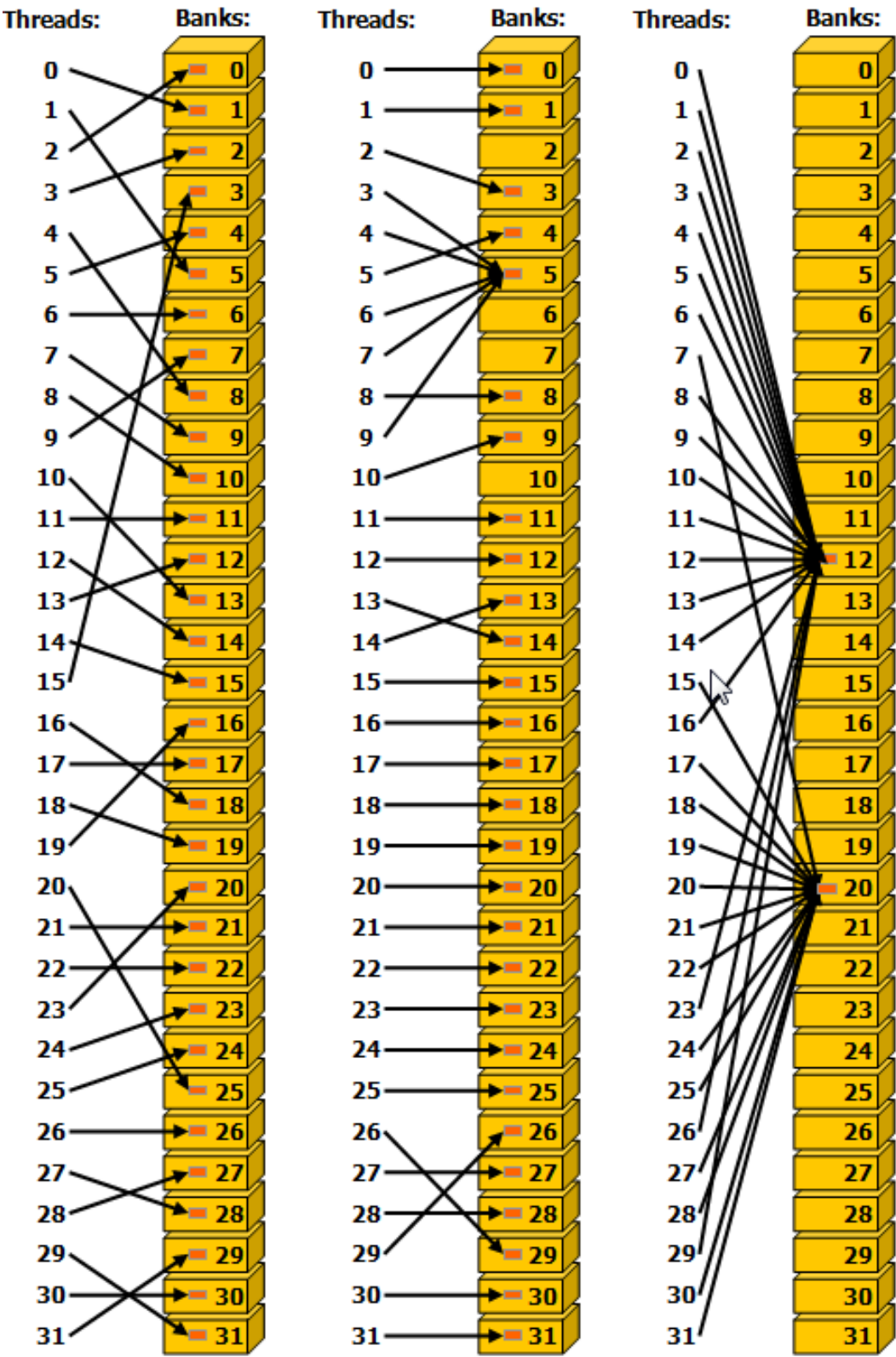


Figure 14: Irregular Shared Memory Accesses.

Left

Conflict-free access via random permutation.

Middle

Conflict-free access since threads 3, 4, 6, 7, and 9 access the same word within bank 5.

Right

Conflict-free broadcast access (threads access the same word within a bank).

Note

Avoiding bank conflicts is an important performance consideration for writing performant CUDA kernels that use shared memory.

2.2.4.2.1. Matrix Transpose Example Using Shared Memory

In the previous example [Matrix Transpose Example Using Global Memory](#), a naive implementation of matrix transpose was illustrated that was functionally correct, but not optimized for efficient use of global memory because the write of the `c` matrix was not coalesced properly. In this example, shared memory will be treated as a user-managed cache to stage loads and stores from global memory, resulting in coalesced global memory access of both reads and writes.

Example**Example with array checks**

```

1  /* definitions of thread block size in X and Y directions */
2
3  #define THREADS_PER_BLOCK_X 32
4  #define THREADS_PER_BLOCK_Y 32
5
6  /* macro to index a 1D memory array with 2D indices in row-major order */
7  /* ld is the leading dimension, i.e. the number of columns in the matrix */
8
9  #define INDX( row, col, ld ) ( ( (row) * (ld) ) + (col) )
10
11 /* CUDA kernel for shared memory matrix transpose */
12
13 __global__ void smem_cuda_transpose(int m, float *a, float *c )
14 {
15
16     /* declare a statically allocated shared memory array */
17
18     __shared__ float smemArray[THREADS_PER_BLOCK_X][THREADS_PER_BLOCK_Y];
19
20     /* determine my row tile and column tile index */
21
22     const int tileCol = blockDim.x * blockIdx.x;
23     const int tileRow = blockDim.y * blockIdx.y;
24
25     /* read from global memory into shared memory array */
26     smemArray[threadIdx.x][threadIdx.y] = a[INDX( tileRow + threadIdx.y, tileCol + threadIdx.x, m )];
27
28     /* synchronize the threads in the thread block */
29     __syncthreads();
30
31     /* write the result from shared memory to global memory */
32     c[INDX( tileCol + threadIdx.y, tileRow + threadIdx.x, m )] = smemArray[threadIdx.y][threadIdx.x];
33     return;
34 }
35 /* end smem_cuda_transpose */

```

The fundamental performance optimization illustrated in this example is to ensure that when accessing global memory, the memory accesses are coalesced properly. Prior to the execution of the copy, each thread computes its `tileRow` and `tileCol` indices. These are the indices for the specific tile that will be operated on, and these tile indices are based on which thread block is executing. Each thread in the `same thread block has the same tileRow and tileCol values`, so it can be thought of as the `starting position of the tile` that this specific thread block will operate on.

The kernel then proceeds with each thread block copying a 32 x 32 tile of the matrix from global memory to shared memory with the following statement. Since the size of a warp is 32 threads, this copy operation will be executed by 32 warps, with no guaranteed order between the warps.

```
smemArray[threadIdx.x][threadIdx.y] = a[INDX( tileRow + threadIdx.y, tileCol + threadIdx.x, m )];
```

Note that because `threadIdx.x` appears in the second argument to `INDX`, consecutive threads are accessing consecutive elements in memory, and the read of `a` is perfectly coalesced.

The next step in the kernel is the call to the `__syncthreads()` function. This ensures that all threads in the thread block have completed their execution of the previous code before proceeding and therefore that the write of `a` into shared memory is completed before the next step. This is critically important because the next step will involve threads reading from shared memory. Without the `__syncthreads()` call, the read of `a` into shared memory would not be guaranteed to be completed by all the warps in the thread block before some warps advance further in the code.

At this point in the kernel, for each thread block, the `smemArray` has a 32 x 32 tile of the matrix, arranged in the same order as the original matrix. To ensure that the elements within the tile are transposed properly, `threadIdx.x` and `threadIdx.y` are swapped when they read `smemArray`. To ensure that the overall tile is placed in the correct place in `c`, the `tileRow` and `tileCol` indices are also swapped when they write to `c`. To ensure proper coalescing, `threadIdx.x` is used in the second argument to `INDX`, as shown by the statement below.

```
c[INDX( tileCol + threadIdx.y, tileRow + threadIdx.x, m )] = smemArray[threadIdx.y][threadIdx.x];
```

This kernel illustrates two common uses of shared memory.

- Shared memory is used to stage data from global memory to ensure that reads from and writes to global memory are both coalesced properly.
- Shared memory is used to allow threads in the same thread block to share data among themselves.

2.2.4.2.2. Shared Memory Bank Conflicts

In [Section 2.2.4.2](#), the bank structure of shared memory was described. In the previous matrix transpose example, the proper coalesced memory access to/from global memory was achieved, but no consideration was given to whether shared memory bank conflicts were present. Consider the following 2d shared memory declaration,

```
__shared__ float smemArray[32][32];
```

Since a warp is 32 threads, each thread in the same warp will have a fixed value for `threadIdx.y` and will have `0 <= threadIdx.x < 32`.

The left panel of [Figure 15](#) illustrates the situation when the threads in a warp access the data in a column of `smemArray`. Warp 0 is accessing memory locations `smemArray[0][0]` through `smemArray[31][0]`. In C++ multi-dimensional array ordering, the last index moves the fastest, so consecutive threads in warp 0 are accessing memory locations that are 32 elements apart. As illustrated in the figure, the colors denote the banks, and this access down the entire column by warp 0 results in a 32-way bank conflict.

The right panel of [Figure 15](#) illustrates the situation when the threads in a warp access the data across a row of `smemArray`. Warp 0 is accessing memory locations `smemArray[0][0]` through `smemArray[0][31]`. In this case, consecutive threads in warp 0 are accessing memory locations that are adjacent. As illustrated in the figure, the colors denote the banks, and this access across the entire row by warp 0 results in no bank conflicts. The ideal scenario is for each thread in a warp to access a shared memory location with a different color.

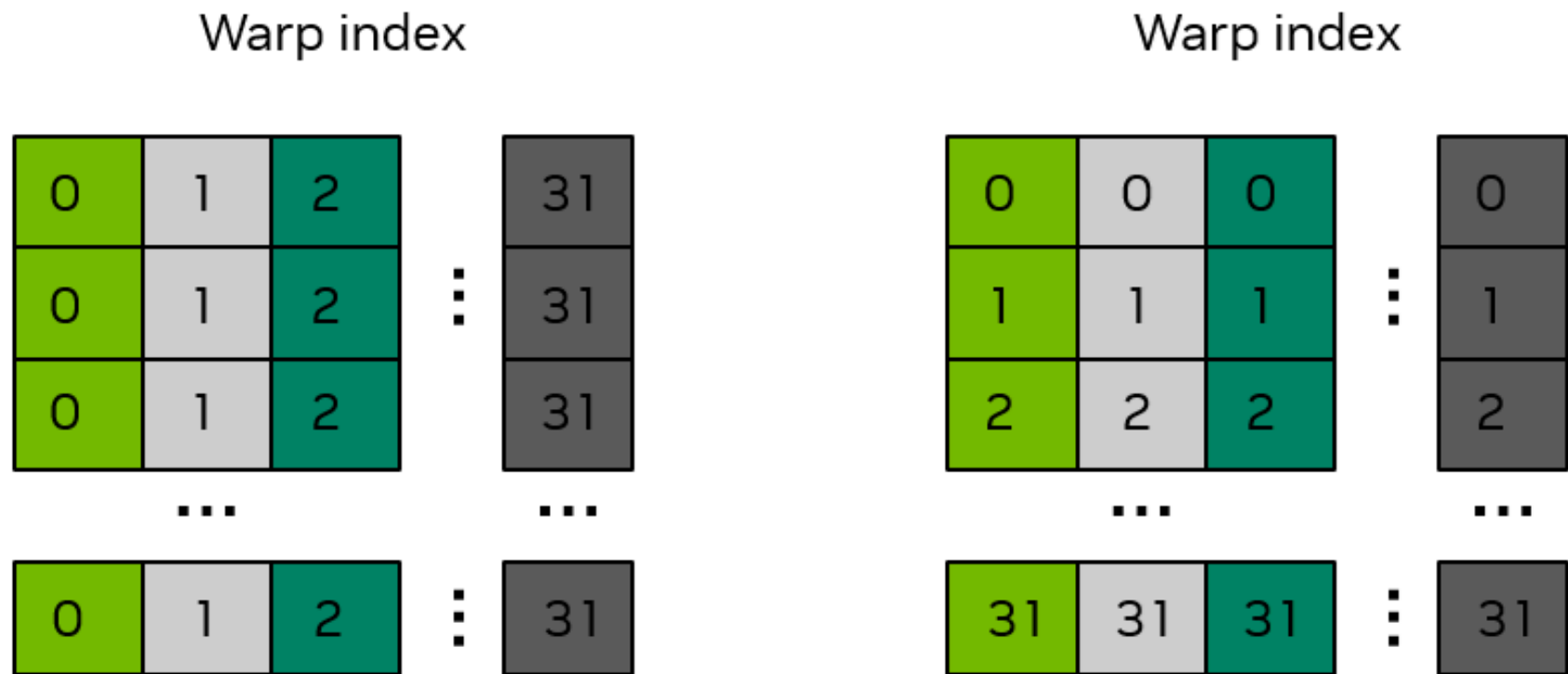


Figure 15: Bank structure in a 32 x 32 shared memory array.

The numbers in the boxes indicate the warp index. The colors indicate which bank is associated with that shared memory location.

Returning to the example from [Section 2.2.4.2.1](#), one can examine the usage of shared memory to determine whether bank conflicts are present. The **first usage of shared memory** is when data from global memory is stored to shared memory:

```
smemArray[threadIdx.x][threadIdx.y] = a[INDX( tileRow + threadIdx.y, tileCol + threadIdx.x, m )];
```

Because **C++ arrays are stored in row-major order**, consecutive threads in the same warp, as indicated by consecutive values of `threadIdx.x`, will access `smemArray` with a stride of 32 elements, because `threadIdx.x` is the first index into the array. **This results in a 32-way bank conflict** and is illustrated by the left panel of [Figure 15](#).

The **second usage** of shared memory is when data from shared memory is written back to global memory:

```
c[INDX( tileCol + threadIdx.y, tileRow + threadIdx.x, m )] = smemArray[threadIdx.y][threadIdx.x];
```

In this case, because `threadIdx.x` is the second index into the `smemArray` array, consecutive threads in the same warp will access `smemArray` with a stride of 1 element. **This results in no bank conflicts** and is illustrated by the right panel of [Figure 15](#).

The matrix transpose kernel as illustrated in [Section 2.2.4.2.1](#) has one access of shared memory that has no bank conflicts and one access that has a 32-way bank conflict. **A common fix to avoid bank conflicts is to pad the shared memory by adding one to the column dimension of the array** as follows:

```
__shared__ float smemArray[THREADS_PER_BLOCK_X][THREADS_PER_BLOCK_Y+1];
```

This minor adjustment to the declaration of `smemArray` will eliminate the bank conflicts. To illustrate this, consider [Figure 16](#) where the shared memory array has been declared with a size of 32 x 33. One observes that whether the threads in the same warp access the shared memory array down an entire column or across an entire row, **the bank conflicts have been eliminated**, i.e., the threads in the same warp access locations with different colors.

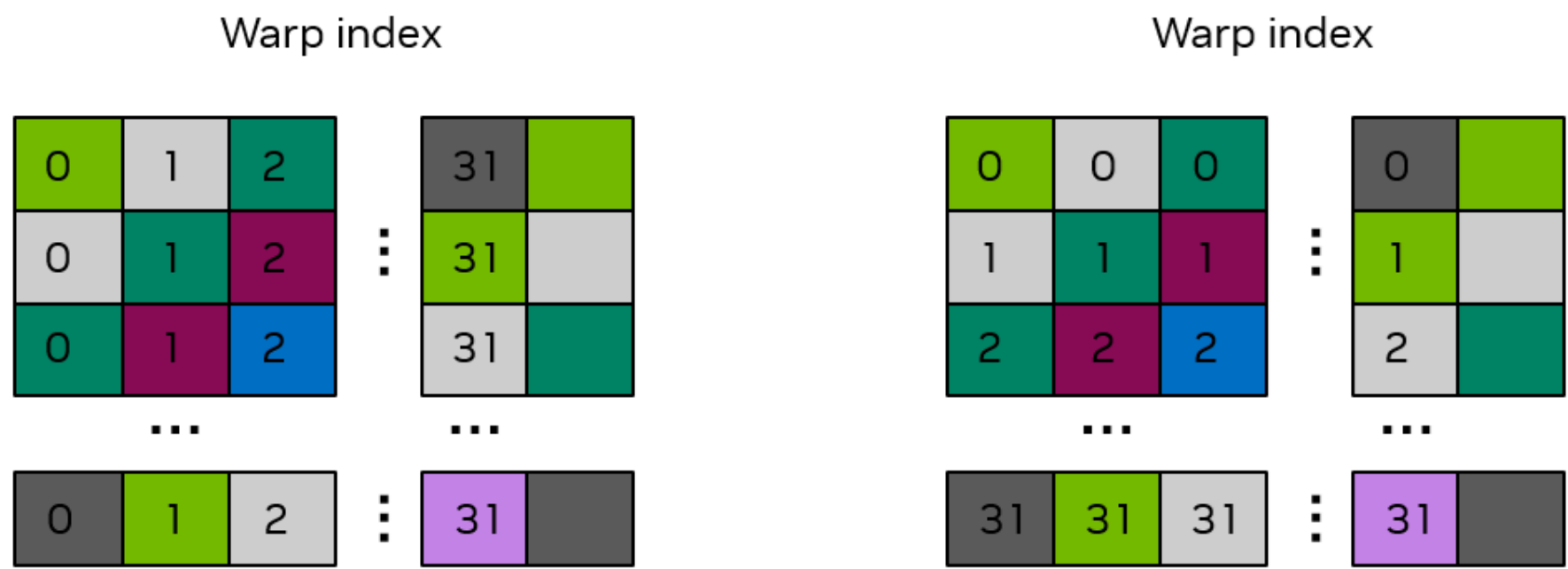


Figure 16: Bank structure in a 32 x 33 shared memory array.

The numbers in the boxes indicate the warp index. The colors indicate which bank is associated with that shared memory location.

2.2.5. Atomics

Performant CUDA kernels rely on expressing as much algorithmic parallelism as possible. The asynchronous nature of GPU kernel execution requires that threads operate as independently as possible. It’s not always possible to have complete independence of threads and as we saw in [Shared Memory](#), there exists a mechanism for threads in the same thread block to exchange data and synchronize.

On the level of an entire grid there is no such mechanism to synchronize all threads in a grid. **There is however a mechanism to provide synchronous access to global memory locations via the use of atomic functions.** Atomic functions allow a thread to obtain a lock on a global memory location and perform a read-modify-write operation on that location. No other thread can access the same location while the lock is held. CUDA provides **atomics with the same behavior as the C++ standard** library atomics as `cuda::std::atomic` and `cuda::std::atomic_ref`. CUDA also provides extended C++ atomics `cuda::atomic` and `cuda::atomic_ref` which allow the user to specify the thread scope of the atomic operation. The details of atomic functions are covered in [Atomic Functions](#).

An example usage of `cuda::atomic_ref` to perform a device-wide atomic addition is as follows, where `array` is an array of floats, and `result` is a float pointer to a location in global memory which is the location where the sum of the array will be stored.

```
__global__ void sumReduction(int n, float *array, float *result) {
    ...
    tid = threadIdx.x + blockIdx.x * blockDim.x;

    cuda::atomic_ref<float, cuda::thread_scope_device> result_ref(result);
    result_ref.fetch_add(array[tid]);
    ...
}
```

Atomic functions should be used sparingly as they enforce thread synchronization that can impact performance.

2.2.6. Cooperative Groups

`Cooperative groups` is a software tool available in CUDA C++ that allows applications to define groups of threads which can synchronize with each other, even if that group of threads spans multiple thread blocks, multiple grids on a single GPU, or even across multiple GPUs. The CUDA programming model in general allows threads within a thread block or thread block cluster to synchronize efficiently, but does not provide a mechanism for specifying thread groups smaller than a thread block or cluster. Similarly, the CUDA programming model does not provide mechanisms or guarantees that enable synchronization across thread blocks.

`Cooperative groups` provide both of these capabilities through software. `Cooperative groups` allows the application to create thread groups that cross the boundary of thread blocks and clusters, though doing so comes with some semantic limitations and performance implications which are described in detail in the [feature section covering cooperative groups](#).

2.2.7. Kernel Launch and Occupancy

When a CUDA kernel is launched, CUDA threads are grouped into thread blocks and a grid based on the execution configuration specified at kernel launch. Once the kernel is launched, the scheduler assigns thread blocks to SMs. The details of which thread blocks are scheduled to execute on which SMs cannot be controlled or queried by the application and no ordering guarantees are made by the scheduler, so programs cannot not rely on a specific scheduling order or scheme for correct execution.

The number of blocks that can be scheduled on an SM depends on the hardware resources a given thread block requires, and the hardware resources available on the SM. When a kernel is first launched, the scheduler begins assigning thread blocks to SMs. As long as SMs have sufficient hardware resources unoccupied by other thread blocks, the scheduler will continue assigning thread blocks to SMs. If at some point no SM has the capacity to accept another thread block, the scheduler will wait until the SMs complete previously assigned thread blocks. Once this happens, SMs are free to accept more work, and the scheduler assigns thread blocks to them. This process continues until all thread blocks have been scheduled and executed.

The `cudaGetDeviceProperties` function allows an application to query the limits of each SM via [device properties](#). Note that there are limits per SM and per thread block.

- `maxBlocksPerMultiProcessor`: The maximum number of resident blocks per SM.
- `sharedMemPerMultiProcessor`: The amount of shared memory available per SM in bytes.
- `regsPerMultiProcessor`: The number of 32-bit registers available per SM.
- `maxThreadsPerMultiProcessor`: The maximum number of resident threads per SM.
- `sharedMemPerBlock`: The maximum amount of shared memory that can be allocated by a thread block in bytes.
- `regsPerBlock`: The maximum number of 32-bit registers that can be allocated by a thread block.
- `maxThreadsPerBlock`: The maximum number of threads per thread block.

The occupancy of a CUDA kernel is the ratio of the number of active warps to the maximum number of active warps supported by the SM. In general, it's a good practice to have occupancy as high as possible which hides latency and increases performance.

To calculate occupancy, one needs to know the resource limits of the SM, which were just described, and one needs to know what resources are required by the CUDA kernel in question. To determine resource usage on a per kernel basis, during program compilation one can use the `--resource-usage` option to `nvcc`, which will show the number of registers and shared memory required by the kernel.

To illustrate, consider a device such as compute capability 10.0 with the device properties enumerated in [Table 2](#).

Table 2: SM Resource Example



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2007-2026, NVIDIA Corporation & affiliates. All rights reserved.

Last updated on Mar 04, 2026.